

PROJECT SUBMISSION

Document Intelligence

A Retrieval-Augmented Generation platform for querying corporate policy documents with grounded answers and citations

Live Application	docintel-omega.vercel.app
GitHub Repository	github.com/bhavikachopra76/assignment_1
API Base URL	docintel-7vvc.onrender.com
Interactive API Docs	docintel-7vvc.onrender.com/docs

Bhavika Chopra · August 2026

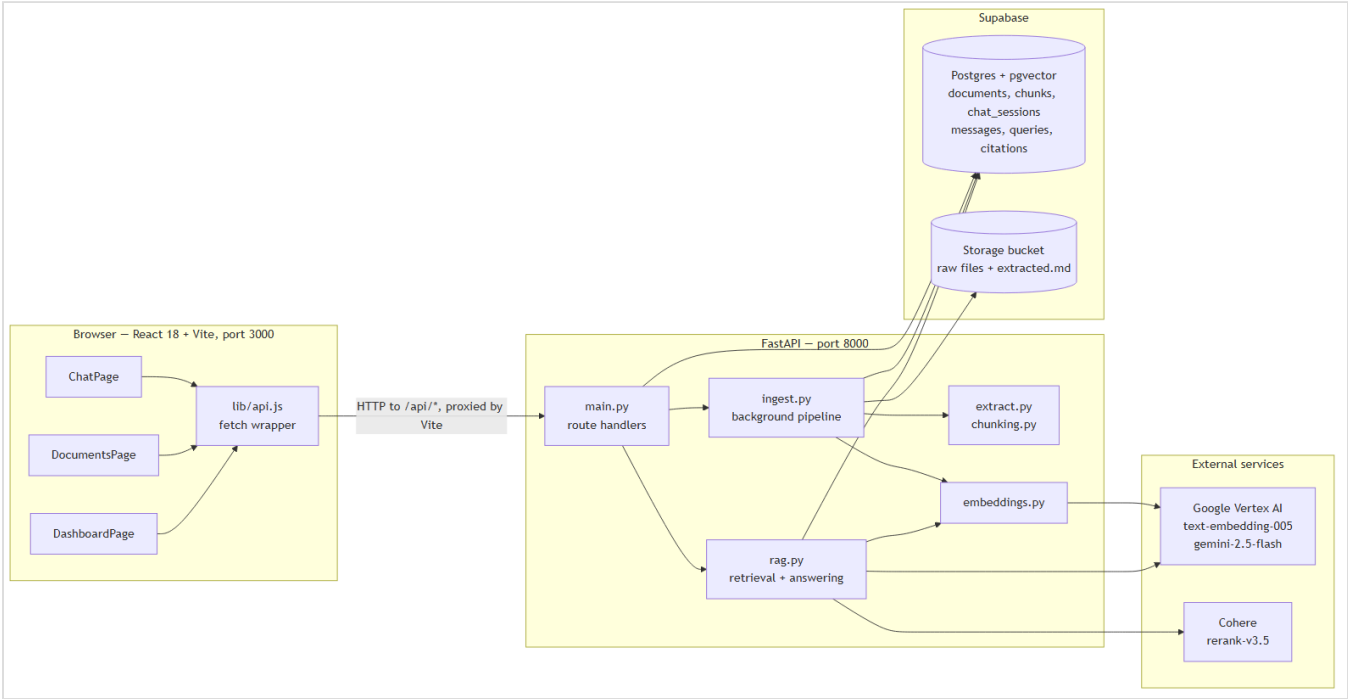
Technology Stack

Component	Technology
Frontend	React 18, Vite, Tailwind CSS — deployed on Vercel
Backend	FastAPI (Python) — deployed on Render
Document parsing	PyMuPDF / pymupdf4llm , python-docx
Chunking	Heading-aware Markdown splitting + pysbd sentence segmentation
Embeddings	Google Vertex AI text-embedding-005 (768 dimensions)
Vector store	Supabase PostgreSQL with pgvector
Retrieval	Hybrid — dense vector similarity + sparse keyword full-text
Reranking	Cohere rerank-v3.5
Generation	Google Gemini gemini-2.5-flash

Part 1 — Architecture

The system has two pipelines that share one storage layer. **Ingestion** turns an uploaded file into searchable vectors. **Querying** answers a question using only those vectors, never outside knowledge.

System Overview



Frontend, API, database and model providers

The frontend never talks to Supabase or the model providers directly. Every call goes to `/api/*` on the backend, which holds all credentials. In development the Vite dev server proxies those calls to `localhost:8000`; in production the frontend is built with the deployed API URL baked in.

Ingestion Pipeline

Uploading returns immediately. The real work runs in a FastAPI `BackgroundTask`, and the frontend polls for progress. This is why the backend is deployed as an always-on container rather than serverless functions — a serverless process is terminated as soon as the response returns, which would kill ingestion partway through embedding.

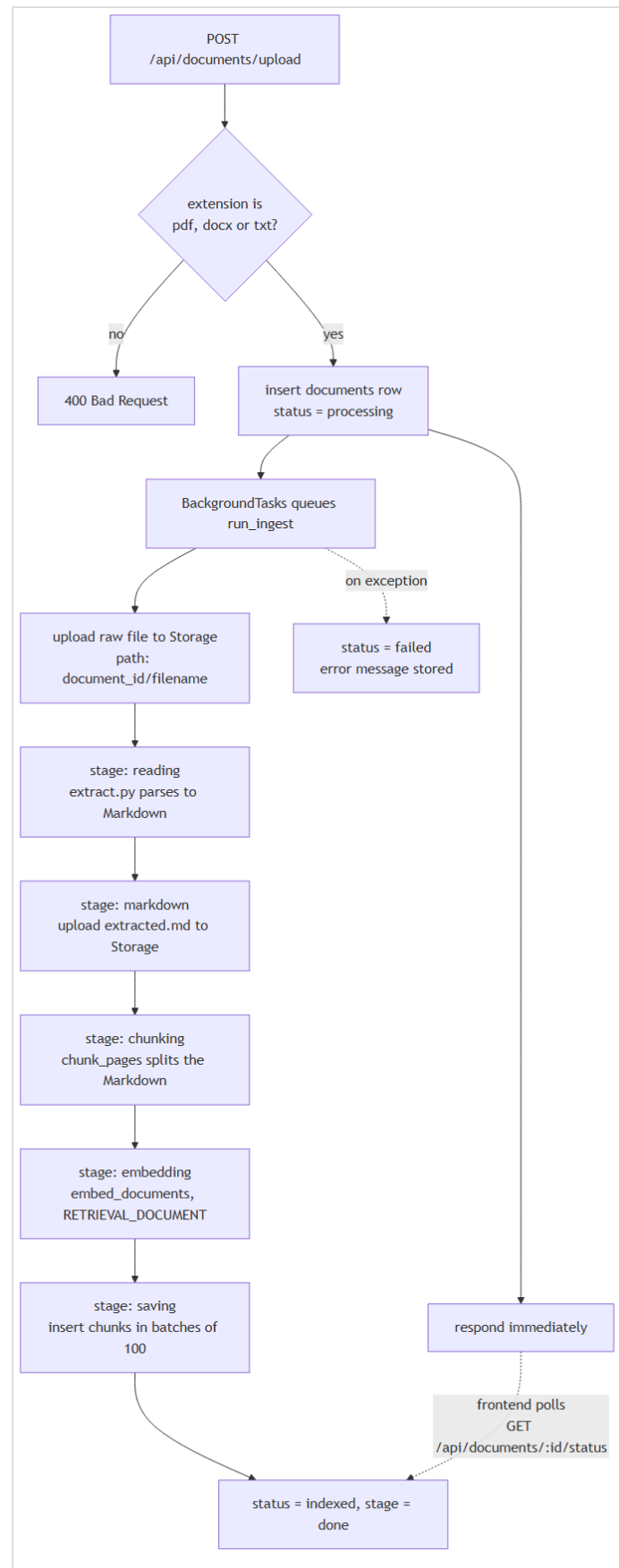
Format	Library	Produces
PDF	<code>pymupdf</code> + <code>pymupdf4llm</code>	Per-page Markdown, author, page count
DOCX	<code>python-docx</code>	Heading styles become <code>#</code> levels; tables become Markdown tables
TXT	standard library	<code>---</code> / <code>===</code> underlines promoted to <code>##</code> headings

All three return the same shape — a list of `(page_number, text)` pairs — so everything downstream is format-agnostic. Only PDFs carry real page numbers, which is why some citations show a page and others do not.

Chunking happens in two passes. First the Markdown is split on headings so a chunk never spans two unrelated sections, with the current heading carried forward as metadata. Then `pysbd` splits on sentence boundaries, accumulating until a chunk reaches roughly 600 tokens and carrying the last 2 sentences forward as overlap. A trailing chunk under 40 tokens is merged back into the previous one.

Each chunk is stored with a 768-dimension embedding. The full-text search column is a *generated* column, so PostgreSQL computes it on insert and keyword search costs no extra write from the application:

```
fts tsvector generated always as (to_tsvector('english', content)) stored
```



Ingestion — upload through extraction, chunking, embedding and indexing

Query Pipeline

Why hybrid retrieval

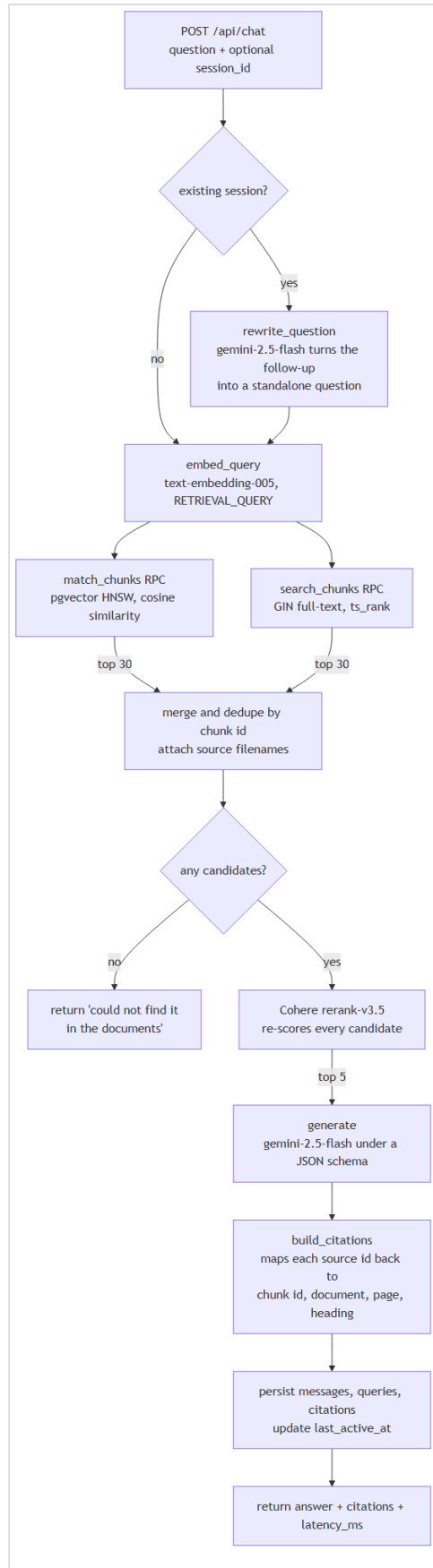
Dense vector search matches meaning but misses exact tokens — policy numbers, allowance amounts, specific job titles. Keyword search catches those but misses paraphrases. Running both and merging by chunk id gives up to 60 candidates with the recall of both methods. A chunk found by both channels appears once.

Why rerank

60 candidates is far more context than the model needs, and once the answer is being written precision matters more than recall. Cohere `rerank-v3.5` scores every candidate against the question directly — a cross-encoder, unlike the bi-encoder embeddings used for retrieval — and only the top 5 reach the prompt.

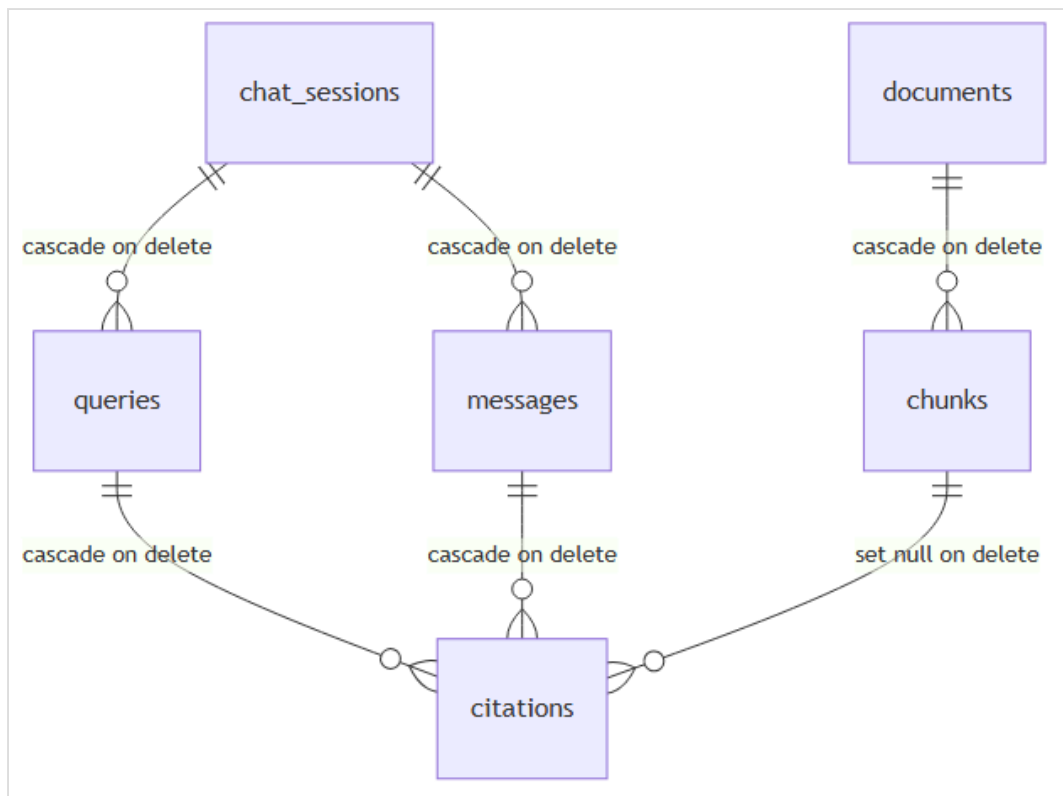
Grounded answering

The prompt numbers each source and requires a fixed JSON response shape. The schema is enforced by Vertex AI's `response_schema`, not merely requested in the prompt. Each returned source id is then validated against the actual candidate list and anything out of range is discarded, so a hallucinated source number cannot produce a citation.



Query — question through hybrid retrieval, reranking and grounded answering

Data Model



Tables and their cascade-delete relationships

Table	Holds
documents	File metadata, ingestion <code>status</code> and <code>stage</code> , page and chunk counts
chunks	Chunk text, heading, page number, 768-d <code>embedding</code> , generated <code>fts</code>
chat_sessions	One row per conversation, ordered by <code>last_active_at</code>
messages	User and assistant turns, used to rebuild a conversation
queries	One row per question with <code>latency_ms</code> , powering the dashboard
citations	Verbatim quote plus a link back to the chunk it came from

Deleting a document cascades to its chunks. Citations keep their stored quote and document name even after the chunk is gone, because `chunk_id` is set to null rather than deleted — so previously given answers stay readable in the dashboard.

Part 2 — API Documentation

Base URL	https://docintel-7vvc.onrender.com
Interactive docs	/docs (Swagger UI) · /redoc
OpenAPI spec	/openapi.json
Local	<code>http://localhost:8000</code>

The Swagger page is executable — a document can be uploaded and a question asked from the browser without writing any client code. The API is hosted on Render's free tier and spins down after 15 minutes of inactivity, so the first request after an idle period takes 50–60 seconds while the server wakes.

Conventions

- All responses are JSON. Request bodies are JSON except the upload endpoint, which is `multipart/form-data`.
- Response fields are **camelCase** while database columns are `snake_case` — `file_type` becomes `type`, `uploaded_at` becomes `uploadDate`.
- Timestamps are ISO 8601 UTC. `uploadDate` is date-only.
- Sizes, latencies and relative times are pre-formatted display strings (`"41 KB"`, `"1.8s"`, `"5 min ago"`), not raw numbers.
- **There is no authentication.** Every endpoint is public. This is a known limitation — the service is scoped as a single-tenant demo, and an API key layer would be the first addition before any real deployment.

Endpoint Summary

Method	Path	Purpose
GET	/	Health check
GET	/api/documents	List all documents
POST	/api/documents/upload	Upload one or more documents
GET	/api/documents/{document_id}/status	Check ingestion progress
DELETE	/api/documents/{document_id}	Delete a document
POST	/api/chat	Ask a question
GET	/api/chat/sessions	List recent chat sessions
GET	/api/chat/sessions/{session_id}/messages	Get messages in a session
DELETE	/api/chat/sessions/{session_id}	Delete a chat session
GET	/api/dashboard/stats	Overall usage statistics
GET	/api/dashboard/queries	Recent questions
GET	/api/dashboard/responses	Recent answers with sources

Documents

GET /api/documents

All uploaded documents, newest first.

```
[
  {
    "id": "5d36f10e-80e3-49fb-8e54-6225fbabae64",
    "filename": "NexaCore_Travel_Policy.docx",
    "type": "DOCX",
    "size": "41 KB",
    "pages": null,
    "chunks": 76,
    "uploadDate": "2026-08-04",
    "status": "Indexed",
    "stage": "done",
    "author": "NexaCore HR",
    "error": null
  }
]
```

`pages` is `null` for DOCX and TXT — only PDFs carry page numbers.

POST /api/documents/upload

Content type `multipart/form-data`, field name `files`, repeated once per file. Accepted extensions: **pdf**, **docx**, **txt**.

```
curl -X POST https://docintel-7vvc.onrender.com/api/documents/upload \
-F "files=@NexaCore_Travel_Policy.docx" \
-F "files=@NexaCore_Employee_Handbook.pdf"
```

200 — returns immediately, before indexing has run:

```
[
  {
    "id": "8c1f2b40-1d3e-4a77-9f21-6b0c5d2e7a13",
    "filename": "NexaCore_Travel_Policy.docx",
    "type": "DOCX",
    "size": "41 KB",
    "pages": null,
    "chunks": 0,
    "uploadDate": "2026-08-05",
    "status": "Processing",
    "stage": null,
    "author": null,
    "error": null
  }
]
```

The response arrives before the document is searchable. **400** if any file has an unsupported extension — the whole request is rejected and nothing is uploaded: `{ "detail": "notes.pptx must be a PDF, DOCX or TXT file" }`

```
GET /api/documents/{document_id}/status
```

Same shape as a single document. Poll while `status` is `Processing`.

status	stage	Meaning
Processing	reading	Extracting text from the file
Processing	markdown	Saving the extracted Markdown
Processing	chunking	Splitting into passages
Processing	embedding	Generating vectors
Processing	saving	Writing chunks to the database
Indexed	done	Ready to query
Failed	<i>last stage reached</i>	<code>error</code> holds the reason

404 — `{ "detail": "Document not found" }`

```
DELETE /api/documents/{document_id}
```

Removes the stored files and the database row. Chunks are removed by cascade.

```
{ "deleted": "5d36f10e-80e3-49fb-8e54-6225fbabae64" }
```

Chat

POST /api/chat

Answers a question using only the uploaded documents.

```
{
  "question": "What is the travel expense reimbursement limit?",
  "session_id": null
}
```

Field	Type	Required	Notes
question	string	yes	Cannot be empty or whitespace
session_id	string null	no	Omit to start a new conversation

Pass `session_id` back on later questions to keep context. When present, the last 6 messages are used to rewrite an ambiguous follow-up ("what about international?") into a standalone question before searching.

```
{
  "sessionId": "9f3c1a02-77b5-4c8e-a1d6-3e0b9c4f2a8d",
  "message": {
    "id": "b21e7d54-9a08-4f13-8c2b-5d7e1f60a934",
    "sender": "assistant",
    "text": "Employees may claim up to $75 per day for meals while travelling domestically.",
    "timestamp": "2026-08-05T11:24:03.512Z",
    "citations": [
      {
        "id": "c47a9e13-2f60-4b85-91d3-8a2c6e0f5b71",
        "documentName": "NexaCore_Travel_Policy.docx",
        "pageNumber": null,
        "section": "Meal Allowances",
        "originalParagraph": "Employees travelling domestically may claim up to $75 per day for meals."
      }
    ]
  }
}
```

`originalParagraph` is copied verbatim from the source passage, so a citation can be checked against the original document. If nothing relevant is found the answer says so and `citations` is empty — the model is instructed never to answer from outside knowledge.

400 empty question · **503** upstream AI rate limit after internal retries. Typical response time is 3–8 seconds; rewriting, embedding, two searches, reranking and generation run in sequence.

GET /api/chat/sessions

The 20 most recently active conversations. `title` is the first question, truncated to 60 characters.

```
[
  {
    "id": "9f3c1a02-77b5-4c8e-a1d6-3e0b9c4f2a8d",
    "title": "What is the travel expense reimbursement limit?",
    "lastActive": "5 min ago"
  }
]
```

GET /api/chat/sessions/{session_id}/messages

Full history for one conversation, oldest first. Same message shape as `POST /api/chat`, with `sender` being `user` or `assistant`. User messages always have empty `citations`. Returns `[]` for an unknown session.

DELETE /api/chat/sessions/{session_id}

Messages, queries and citations are removed by cascade, so the session's questions also disappear from dashboard analytics. **404** if the chat does not exist.

Dashboard

GET /api/dashboard/stats

```
{
  "totalDocuments": 3,
  "totalChunks": 326,
  "totalQuestions": 7,
  "avgResponseTime": "24.2s",
  "documentGrowth": "+3 this week",
  "chunkGrowth": "+326 chunks",
  "questionGrowth": "+2 today",
  "timeTrend": "over 7 queries"
}
```

The growth and trend fields are display strings for the stat cards, not values to compute with.

GET /api/dashboard/queries

The last 50 questions asked, with how long each took.

```
[
  {
    "id": "e83b6c19-4d2a-4f70-b5e1-9c7a3f08d264",
    "question": "What is the travel expense reimbursement limit?",
    "time": "5 min ago",
    "status": "Completed",
    "latency": "1.8s"
  }
]
```

GET /api/dashboard/responses

The last 50 answers, each labelled with the first document it cited.

```
[
  {
    "id": "e83b6c19-4d2a-4f70-b5e1-9c7a3f08d264",
    "question": "What is the travel expense reimbursement limit?",
    "answer": "Employees may claim up to $75 per day for meals while travelling domestically.",
    "source": "NexaCore_Travel_Policy.docx",
    "timestamp": "2026-08-05T11:24:03.512Z"
  }
]
```

`source` includes a page when one exists (`"Handbook.pdf (Page 12)"`) and is `"-"` when the answer had no citations.

Errors

Every error uses FastAPI's standard shape: `{ "detail": "Document not found" }`

Code	When
400	Empty question, or an unsupported file extension
404	Document or chat session does not exist
422	Request body failed validation
503	Upstream AI rate limit, after internal retries

Rate limits are retried before surfacing: embedding retries 5 times with 20-second waits, generation retries 3 times backing off 10 then 20 seconds. A `503` means those were exhausted. Failures during *ingestion* never surface as HTTP errors, because upload returns before that work runs — they appear as `status: "Failed"` with a reason in `error` on the document.

External Services

Service	Used for	Model / feature
Google Vertex AI	Embedding chunks and queries	<code>text-embedding-005</code> , 768 dimensions
Google Vertex AI	Question rewriting and grounded answering	<code>gemini-2.5-flash</code> with an enforced JSON response schema
Cohere	Reranking retrieved candidates	<code>rerank-v3.5</code> , 60 candidates down to top 5
Supabase	Postgres with pgvector, plus file storage	HNSW cosine index, GIN full-text index

Chunks are embedded with task type `RETRIEVAL_DOCUMENT` and questions with `RETRIEVAL_QUERY` — the same model, but asymmetric task types produce better matching between short questions and longer passages.